

Mid- Term Examination

Fourth Semester, May 2023

Database Management System

CIC-210

(Please write your Enrolment No. immediately)

Enrolment No. _____

MID TERM EXAMINATION

B.TECH PROGRAMMES (UNDER THE AEGIS OF USICT)

Fourth Semester, May, 2023

Paper Code: CIC-210

Subject: Database Management System

Time: 1½ Hrs.

Max. Marks: 30

Note: Attempt Q. No. 1 which is compulsory and any two more questions from remaining.

Q. No.	Question	Max. Marks	CO(s)
1	Attempt the following:		
(a)	What are the characteristics of database system?	2	CO 1
(b)	What is data independence? Explain different types of data independence.	2	CO 1
(c)	Compare and contrast between relational algebra and relation calculus.	2	CO 2
(d)	Discuss different types of joins with the help of example.	2	CO 2
(e)	Explain any two DDL and two DML commands with example.	2	CO 2
2	Consider a university which has several schools. Each school is headed by a teacher and offers one or more courses. A course can be offered by only one school. A course may be full time course or part time course. Each course consists of a set of subjects. Each school may have one or more teachers. A teacher can teach one or more subjects. The student can get admission in only one course. You may assume appropriate attributes of each entity type and relationship type. State assumptions clearly, if any. You are required to: (a) Prepare an ER Diagram using generation-specialization hierarchy and/or aggregation if required. (b) Derive the relational schema from the above prepared ER-Diagram.	5+5	CO 1
3	Consider the following relational schema: Hotel (Hotel_No, Name, City) Room (Room_No, Hotel_No, Type, Price) Guest (Guest_No, Name, City) Booking (Hotel_No, Guest_No, Date_From, Date_To, Room_No) Write SQL Command and Relational Algebraic Expression for the following queries: (a) List the name and address of guests belongs to Delhi and Mumbai. (b) Find out the total number hotels in each city. (c) List the all bookings for hotel 'ITC Maurya' by the guest 'G010' using subquery. (d) Find the total daily revenue from all the double rooms. (e) List Hotel Name, Room No, Type and Price of all rooms.	5 x 2	CO2
4 (a)	Explain three-schema architecture with the help of diagram.	5	CO 1
(b)	What are integrity constraints? Explain different integrity constraints with the help of examples.	5	CO 2

Q 1 (a) What are the characteristics of database system?

Answer:

A database management system (DBMS) is a collection of programs that enables users to create and maintain a database. The DBMS is hence a general-purpose software system that facilitates the processes of defining, constructing, and manipulating databases for various applications.

Traditionally, data was organized in file formats. DBMS was a new concept then, and all the research was done to make it overcome the deficiencies in traditional style of data management.

A modern DBMS has the following characteristics –

1. **Real-world entity** – A modern DBMS is more realistic and uses real-world entities to design its architecture. It uses the behavior and attributes too. For example, a school database may use students as an entity and their age as an attribute.
2. **Relation-based tables** – DBMS allows entities and relations among them to form tables. A user can understand the architecture of a database just by looking at the table names.
3. **Isolation of data and application** – A database system is entirely different than its data. A database is an active entity, whereas data is said to be passive, on which the database works and organizes. DBMS also stores metadata, which is data about data, to ease its own process.
4. **Less redundancy** – DBMS follows the rules of normalization, which splits a relation when any of its attributes is having redundancy in values. Normalization is a mathematically rich and scientific process that reduces data redundancy.
5. **Consistency** – Consistency is a state where every relation in a database remains consistent. There exist methods and techniques, which can detect attempt of leaving database in inconsistent state. A DBMS can provide greater consistency as compared to earlier forms of data storing applications like file-processing systems.
6. **Query Language** – DBMS is equipped with query language, which makes it more efficient to retrieve and manipulate data. A user can apply as many and as different filtering options as required to retrieve a set of data. Traditionally it was not possible where file-processing system was used.
7. **ACID Properties** – DBMS follows the concepts of Atomicity, Consistency, Isolation, and Durability (normally shortened as ACID). These concepts are applied on transactions, which manipulate data in a database. ACID properties help the database stay healthy in multi-transactional environments and in case of failure.
8. **Multiuser and Concurrent Access** – DBMS supports multi-user environment and allows them to access and manipulate data in parallel. Though there are restrictions on transactions when users attempt to handle the same data item, but users are always unaware of them.
9. **Multiple views** – DBMS offers multiple views for different users. A user who is in the Sales department will have a different view of database than a person working in the Production department. This feature enables the users to have a concentrate view of the database according to their requirements.

10. **Security** – Features like multiple views offer security to some extent where users are unable to access data of other users and departments. DBMS offers methods to impose constraints while entering data into the database and retrieving the same at a later stage. DBMS offers many different levels of security features, which enables multiple users to have different views with different features. For example, a user in the Sales department cannot see the data that belongs to the Purchase department. Additionally, it can also be managed how much data of the Sales department should be displayed to the user. Since a DBMS is not saved on the disk as traditional file systems, it is very hard for miscreants to break the code.

(b) What is data independence? Explain different types of data independence.

Answer:

The three-schema architecture can be used to explain the concept of data independence, which can be defined as the capacity to change the schema at one level of a database system without having to change the schema at the next higher level.

We can define two types of data independence:

1. **Logical data independence** : It is the capacity to change the conceptual schema without having to change external schemas or application programs. We may change the conceptual schema to expand the database (by adding a record type or data item), or to reduce the database (by removing a record type or data item). In the latter case, external schemas that refer only to the remaining data should not be affected. Only the view definition and the mappings need be changed in a DBMS that supports logical data independence. Application programs that reference the external schema constructs must work as before, after the conceptual schema undergoes a logical reorganization. Changes to constraints can be applied also to the conceptual schema without affecting the external schemas or application programs.
2. **Physical data independence** : It is the capacity to change the internal schema without having to change the conceptual (or external) schemas. Changes to the internal schema may be needed because some physical files had to be reorganized - for example, by creating additional access structures - to improve the performance of retrieval or update. If the same data as before remains in the database, we should not have to change the conceptual schema.

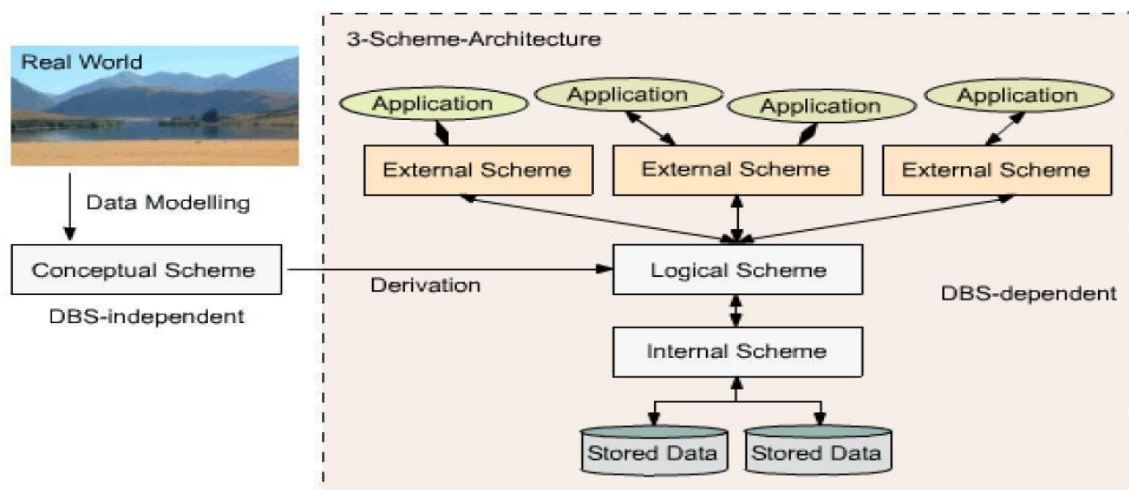


Figure 1. Three-Schemes Architecture

(c) Compare and Contrast between relational algebra and relational calculus.

Answer:

Difference between Relational Algebra and Relational Calculus:

S.NO	Basis of Comparison	Relational Algebra	Relational Calculus
1.	Language Type	It is a Procedural language.	Relational Calculus is a Declarative (non-procedural) language.
2.	Procedure	Relational Algebra means how to obtain the result.	Relational Calculus means what result we have to obtain.
3.	Order	In Relational Algebra, the order is specified in which the operations have to be performed.	In Relational Calculus, the order is not specified.
4.	Domain	Relational Algebra is independent of the domain.	Relation Calculus can be domain-dependent because of domain relational calculus.
5.	Programming language	Relational Algebra is nearer to a programming language.	Relational Calculus is not nearer to programming language but to natural language.
6.	Inclusion in SQL	The SQL includes only some features from the relational algebra.	SQL is based to a greater extent on the tuple relational calculus.
7.	Relationally completeness	Relational Algebra is one of the languages in which queries can be expressed but the queries should also be expressed in relational calculus to be relationally complete.	For a database language to be relationally complete, the query written in it must be expressible in relational calculus.

8.	Query Evaluation	The evaluation of the query relies on the order specification in which the operations must be performed.	The order of operations does not matter in relational calculus for the evaluation of queries.
9.	Database access	For accessing the database, relational algebra provides a solution in terms of what is required and how to get that information by following a step-by-step description.	For accessing the database, relational calculus provides a solution in terms as simple as what is required and lets the system find the solution for that.
10.	Expressiveness	The expressiveness of any given language is judged using relational algebra operations as a standard.	The completeness of a language is measured in the manner that it is least as powerful as calculus. That implies relation defined using some expression of the calculus is also definable by some other expression, the language is in question.

(d) Discuss different types of Joins with examples.

Answer:

Join in DBMS is a binary operation which allows you to combine join product and selection in one single statement. The goal of creating a join condition is that it helps you to combine the data from two or more DBMS tables. The tables in DBMS are associated using the primary key and foreign keys.

Types of Join

There are mainly two types of joins in DBMS:

- 1. Inner Joins: Theta, Natural, EQUI**
- 2. Outer Join: Left, Right, Full**

I. Inner Join

Inner Join is used to return rows from both tables which satisfy the given condition. It is the most widely used join operation and can be considered as a default join-type. An Inner join or equijoin is a comparator-based join which uses equality comparisons in the join-predicate. However, if you use other comparison operators like ">" it can't be called equijoin.

Inner Join further divided into three subtypes:

1. Theta join
2. Natural join
3. EQUI join

1. Theta Join

Theta Join allows you to merge two tables based on the condition represented by theta. Theta joins work for all comparison operators. It is denoted by symbol θ . The general case of JOIN operation is called a Theta join.

Syntax:

$A \bowtie_{\theta} B$

Theta join can use any conditions in the selection criteria.

Table A			Table B	
column 1	column 2		column 1	column 2
1	1		1	1
1	2		1	3

For example:

$A \bowtie_{A.column\ 2 > B.column\ 2} (B)$

$A \bowtie_{A.column\ 2 > B.column\ 2} (B)$	
column 1	column 2
1	2

2. EQUI Join

EQUI Join is done when a Theta join uses only the equivalence condition. EQUI join is the most difficult operation to implement efficiently in an RDBMS, and one reason why RDBMS have essential performance problems.

For example:

$A \bowtie_{A.column\ 2 = B.column\ 2} (B)$

$A \bowtie_{A.column\ 2 = B.column\ 2} (B)$	
column 1	column 2
1	1

3. Natural Join ($\bowtie$)

Natural Join does not utilize any of the comparison operators. In this type of join, the attributes should have the same name and domain. In Natural Join, there should be at least one common

attribute between two relations. It performs selection forming equality on those attributes which appear in both relations and eliminates the duplicate attributes.

Example:

Consider the following two tables

C	
Num	Square
2	4
3	9

D	
Num	Cube
2	8
3	18

C ⋈ D

Num	Square	Cube
2	4	8
3	9	18

II. Outer Join

An Outer Join doesn't require each record in the two join tables to have a matching record. In this type of join, the table retains each record even if no other matching record exists.

Three types of Outer Joins are:

1. **Left Outer Join**
2. **Right Outer Join**
3. **Full Outer Join**

1. Left Outer Join (A ⋈_L B)

Left Outer Join returns all the rows from the table on the left even if no matching rows have been found in the table on the right. When no matching record is found in the table on the right, NULL is returned.

Consider the following 2 Tables

A	
Num	Square
2	4
3	9
4	16

B	
Num	Cube
2	8
3	18
5	75

(A $\bowtie$ B)

Num	Square	Cube
2	4	8
3	9	18
4	16	—

2. Right Outer Join (A $\bowtie$ B)

Right Outer Join returns all the columns from the table on the right even if no matching rows have been found in the table on the left. Where no matches have been found in the table on the left, NULL is returned. RIGHT outer JOIN is the opposite of LEFT JOIN.

For e.g., let's assume that you need to get the names of members and movies rented by them. Now we have a new member who has not rented any movie yet.

A $\bowtie$ B

Num	Cube	Square
2	8	4
3	18	9
5	75	—

3. Full Outer Join (A $\bowtie$ B)

In a Full Outer Join , all tuples from both relations are included in the result, irrespective of the matching condition.

Example:

(A $\bowtie$ B)

Num	Square	Cube
2	4	8
3	9	18
4	16	—
5	—	75

(e) Explain any two DDL and DML commands with example.

Answer:

SQL Commands

- SQL commands are instructions. It is used to communicate with the database. It is also used to perform specific tasks, functions, and queries of data.
- SQL can perform various tasks like create a table, add data to tables, drop the table, modify the table, set permission for users.

Types of SQL Commands

There are five types of SQL commands: DDL, DML, DCL, TCL, and DQL.

1. Data Definition Language (DDL)

- DDL changes the structure of the table like creating a table, deleting a table, altering a table, etc.
- All the command of DDL are auto-committed that means it permanently save all the changes in the database.

Some commands that come under DDL:

- CREATE
- ALTER
- DROP
- TRUNCATE

a. **CREATE** It is used to create a new table in the database.

Syntax:

```
CREATE TABLE TABLE_NAME (COLUMN_NAME DATATYPES[,....]);
```

Example:

```
CREATE TABLE EMPLOYEE(Name VARCHAR2(20), Email VARCHAR2(100), DOB DATE);
```

b. DROP: It is used to delete both the structure and record stored in the table.

Syntax

```
DROP TABLE table_name;
```

Example

```
DROP TABLE EMPLOYEE;
```

c. ALTER: It is used to alter the structure of the database. This change could be either to modify the characteristics of an existing attribute or probably to add a new attribute.

Syntax:

To add a new column in the table

```
ALTER TABLE table_name ADD column_name COLUMN-definition;
```

To modify existing column in the table:

```
ALTER TABLE table_name MODIFY(column_definitions....);
```

EXAMPLE

```
ALTER TABLE STU_DETAILS ADD(ADDRESS VARCHAR2(20)); ALTER TABLE STU_DETAILS MODIFY (NAME VARCHAR2(20));
```

d. TRUNCATE: It is used to delete all the rows from the table and free the space containing the table.

Syntax:

```
TRUNCATE TABLE table_name;
```

Example:

```
TRUNCATE TABLE EMPLOYEE;
```

2. Data Manipulation Language

- DML commands are used to modify the database. It is responsible for all form of changes in the database.
- The command of DML is not auto-committed that means it can't permanently save all the changes in the database. They can be rollback.

Here are some commands that come under DML:

- INSERT
- UPDATE
- DELETE

a. INSERT: The INSERT statement is a SQL query. It is used to insert data into the row of a table.

Syntax:

```
INSERT INTO TABLE_NAME (col1, col2, col3,... col N) VALUES (value1, value2, value3, .... valueN);
```

Or

```
INSERT INTO TABLE_NAME VALUES (value1, value2, value3, .... valueN);
```

For example:

```
INSERT INTO javatpoint (Author, Subject) VALUES ("Sonoo", "DBMS");
```

b. UPDATE: This command is used to update or modify the value of a column in the table.

Syntax:

```
UPDATE table_name SET [column_name1= value1,...column_nameN = valueN] [WHERE CONDITION]
```

For example:

```
UPDATE students SET User_Name = 'Sonoo' WHERE Student_Id = '3'
```

c. DELETE: It is used to remove one or more row from a table.

Syntax:

```
DELETE FROM table_name [WHERE condition];
```

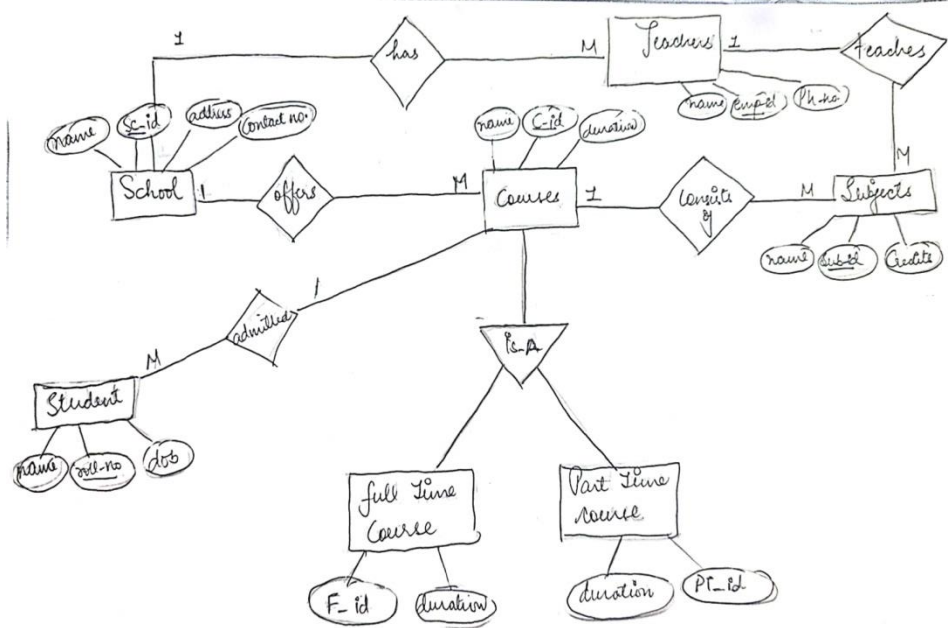
For example:

```
DELETE FROM javatpoint  
WHERE Author="Sonoo";
```

Q2 (a) Consider a university which has several schools. Each school is headed by a teacher and offers one or more courses. A course can be offered by only one school. A course can be offered by only one school. A course may be full time course or part time course. Each course consists of a set of subjects. Each school may have one or more teachers. A teacher can teach one or more subjects. The student can get admission in only one course. You may assume appropriate attributes of each entity type and relationship type. State assumptions clearly, if any.

You are required to:

- (a) Prepare an ER Diagram using generation-specialization hierarchy and/or aggregation if required.
- (b) Derive the relational schema from the above prepared ER-Diagram.



Schema derived from Strong Entity set.

- 1) School = (name, sc-id, address, contact-no)
- 2) Courses = (name, c-id, duration)
- 3) Subjects = (name, sub-id, credits)
- 4) Teachers = (name, emp-id, ph-no)
- 5) Student = (name, roll-no, dob)

Schemas derived from Relationship sets

- 6) offers = (sc-id, c-id)
- 7) consists of = (c-id, sub-id)
- 8) has = (sc-id, emp-id)
- 9) teaches = (emp-id, sub-id)
- 10) admitted = (c-id, roll-no)

Schema derived from an ISA relationships

- 11) Full-Time-Course = (c-id, F-id, duration)
- 12) Part-Time-Course = (c-id, PT-id, duration)

Q3 Consider the following relational schema:

Hotel(Hotel_no, Name, City)

Room(Room_no, Hotel_No, Type, Price)

Guest(Guest_No, Name, City)

Booking(Hotel_no, Guest_No, Date_from, Date_to, Room_no)

Write sql command and relational algebraic expression

a) List the name and address of guests belong to Delhi and Mumbai

a) SQL Command

```
SELECT Name, City
FROM Guest
WHERE City = 'Delhi' OR City = 'Mumbai';
```

Relational Algebra

π Name, City (σ (city = 'Delhi' $\vee$ city = 'Mumbai')) (Guest) ;

b) Find out the total number hotels in each city.

b) SQL Command

```
SELECT City, COUNT(DISTINCT Hotel_NO) AS total_hotels
FROM hotels
GROUP BY City;
```

Relational Algebra

σ (city) (G COUNT-DISTINCT (Hotel_No) as total_hotels) (Hotel));

c) List all booking for hotel ITC Maurya by the guest 'G010' using subquery.

SQL Command

```
SELECT * FROM Booking
WHERE Hotel_no = (SELECT Hotel_No FROM Hotel WHERE Name = 'ITC Maurya')
AND Guest_No = 'G010';
```

Relational Algebra

$(\sigma \text{ Hotel_no} = (\pi \text{ Hotel_no} (\sigma \text{ Name} = \text{'ITC Maurya'} (\text{Hotel}))) \wedge \text{Guest_No} = \text{'G010'})$
(Booking))

d) Find the total daily revenue from all double rooms

d) SQL Command

```
SELECT SUM(Price) AS total_daily_revenue
FROM Room
WHERE Type = 'Double';
```

Relational Algebra

π sum(Price) as total_daily_revenue (σ (type) = 'Double') (Room)

e) List hotel name, room no, type and price of all rooms

SQL Command

```
SELECT Hotel.Name, Room.Room_No, Room.Type, Room.Price  
FROM Room NATURAL JOIN Hotel;
```

Relational Algebra

π Hotel.Name, Room.Room_No, Room.Type, Room.Price (Room $\bowtie$ Hotel)

(4)(a) Explain Three Schema Architecture with example.

Answer:

Following are the three levels of database architecture,

1. Physical Level
2. Conceptual Level
3. External Level

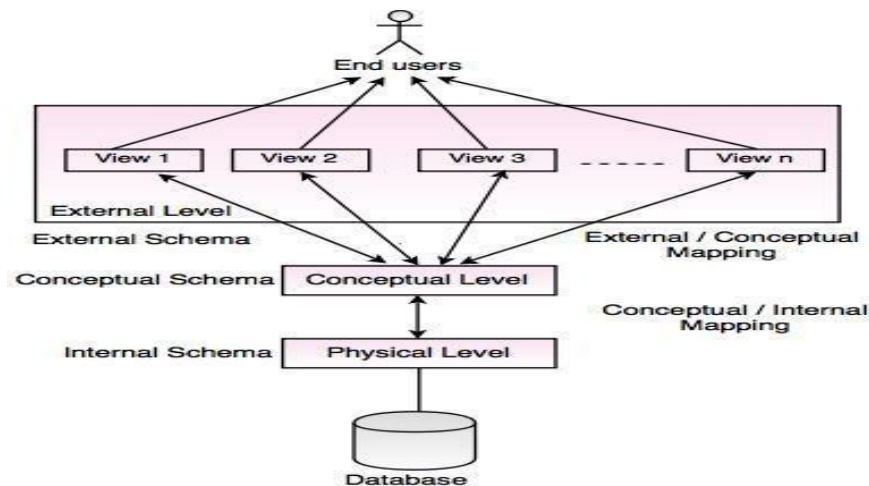


Fig. Three Level Architecture of DBMS

- Mapping is the process of transforming request response between various database levels of architecture. Mapping is not good for small database, because it takes more time.
- In External / Conceptual mapping, DBMS transforms a request on an external schema against the conceptual schema.
- In Conceptual / Internal mapping, it is necessary to transform the request from the conceptual to internal levels.

1. Physical Level:

- Physical level describes the physical storage structure of data in database.
- It is also known as Internal Level.
- This level is very close to physical storage of data.
- At lowest level, it is stored in the form of bits with the physical addresses on the secondary storage device.

- At highest level, it can be viewed in the form of files.
- The internal schema defines the various stored data types. It uses a physical data model.

2. Conceptual Level:

- Conceptual level describes the structure of the whole database for a group of users. It is also called as the data model.
- Conceptual schema is a representation of the entire content of the database.
- These schema contains all the information to build relevant external records.
- It hides the internal details of physical storage.

3. External Level:

- External level is related to the data which is viewed by individual end users.
- This level includes a no. of user views or external schemas. This level is closest to the user.
- External view describes the segment of the database that is required for a particular user group and hides the rest of the database from that user group.

(4) (b) What are integrity constraints? Explain different integrity constraints with the help of examples.

Answer:

In DBMS, Integrity constraints can be defined as a set of rules that are used to maintain the information's quality. This ensures that the data integration is not affected at all by data insertion, updation or other processes. Hence integrity constraints are like insurance to guard the database if there is any accidental damage.

Types of Integrity Constraints

- 1. Domain constraint**
- 2. Entity integrity constraint**
- 3. Referential integrity constraint**
- 4. Key constraint**

1. Domain constraint

In DBMS, Domain constraints can be defined as a set of values that are valid for an attribute. The domain's data type includes string, character, integer, time etc. The value must be in the corresponding domain of the attribute.

Example

Name	Semester	Age
------	----------	-----

Adarsh	1st	19
Kushal	4th	20
Akash	3rd	20
Vishal	7th	H

Now in the table above, we can observe in the column Age, the data type of the domain is an integer but the attributes data type is a character.

2. Entity integrity constraint

This constraint states that in DBMS we can not make the primary key with the value NULL. Now, this is because if the primary key is NULL, then we won't be able to determine or identify the tuple in the relation. But in a relation, there can be NULL values but they must be not the primary key.

Example

Name	Semester	Age
Adarsh	1st	19
Kushal	4th	20
Akash	3rd	20
Vishal	7th	H

In the above example, S_No column and last tuple, the attribute is null, so it can not be assigned as the primary key.

3. Referential integrity constraint

This constraint is defined between two tables. Let us consider tables A and B, so in this constraint, if a foreign key is referred to as a primary key of another table then the contents of the foreign key of table A must be null or available to table B.

Example

TABLE 1

S_No	Name	Semester	Code
1	Adarsh	1st	1258
2	Kushal	4th	2451
3	Akash	3rd	5758

4	Vishal	7th	9654
5	Sankalp	5th	4178

TABLE 2

Code	City
4178	Patna
2451	Delhi
9654	Mumbai
1258	Pune

Now in the example above, we can see that in table 1, Code 5758 is not valid, as this attribute is not defined in table 2 and also the attribute is assigned as the primary key, and also Code in table 1 is assigned the foreign key.

4. Key constraint

In DBMS, a key is used to uniquely identify an entity in an entity set. An entity set can have multiple keys but out only one can be a primary key. This primary key should not be null and must be unique. Although it can contain a null and a non-null unique value.

Example

S. No	Name	Semester	Code
1	Adarsh	1st	1258
2	Kushal	4th	2451
3	Akash	3rd	5758
4	Vishal	7th	9654
4	Sankalp	5th	4178

In the example above, S_No can not be defined as a primary key because it contains a duplicate value. All the rows must be unique.